ParaDEVS Library

How-To Guide and Tutorial (library v0)

ParaDEVS Library workspace, Binghamton

October 9, 2026

Abstract

The ParaDEVS library runs an ordinary MS4 DEVS model as a *paratemporal* simulation. Each time the model makes a random choice, the engine follows every outcome (or a sample of them) and weights it by its probability. The result is the full probability distribution of what you choose to observe, rather than one random run. This guide shows how to set up and run the library, then gives two tutorials: (1) building a small DEVS model from scratch in Java and testing it against a hand-computed answer, and (2) bringing a model over from an MS4 Me project. It ends with an API reference and a troubleshooting table. Every program output shown here was produced by the code in the library folder.

ParaDEVS documents. The entry point is **Start Here**, a reading guide that links every document.

- **Build Report**: what has been built, how it was verified, what is next
- **How-To Guide** (this document): setup, tutorials, API reference, troubleshooting
- **Architecture Guide**: how the library works and how to read results
- **RL Plan**: reinforcement-learning plan, analysis and results

Contents

1 What the library does **3**

2 Setup **3**

2.1 Requirements . . . 3

2.2 Folder layout . . . 4

2.3 Build and run the tests . . . 4

2.4 Run your own model . . . 4

2.5 Working in VS Code . . . 5

3 Key ideas in one page **6**

4 Tutorial 1: a DEVS model built purely in Java **7**

4.1 The answer by hand . . . 7

4.2 Step 1: write the atomic model . . . 7
4.3 Step 2: run it with an input trace . . . 9
4.4 Step 3: make it a coupled model . . . 9
4.5 Step 4: options . . . 11
4.6 Step 5: it is still an ordinary MS4 model . . . 11
4.7 Step 6: test it . . . 11

5 Tutorial 2: bringing a model over from MS4 Me 12
5.1 What to bring over . . . 12
5.2 Pick a build route . . . 12
5.3 The edits . . . 13
5.4 Worked example: JobProcessorExample . . . 13
5.5 Port names in generated code . . . 14
5.6 Keeping MS4 Me in sync . . . 15
5.7 Testing an imported model . . . 15

6 API reference 15
6.1 Model side . . . 15
6.2 Experiment side: `ParaSpec` . . . 16
6.3 Results: `ParaResult` . . . 16

7 Troubleshooting 17

8 Scope of v0 and further reading 17

1 What the library does

A stochastic DEVS model draws random values: a service time, a hit type, a trading action. An ordinary run draws one value each time and produces one trajectory. To get the distribution of an outcome, you would normally run the model many times.

The paratemporal engine instead *branches*: when the model asks for a random value with k possible outcomes, the engine makes k copies of the run. Each copy continues with one outcome and carries a weight multiplied by that outcome's probability. When a copy finishes, its weight and what it observed are added to the result. With exhaustive branching the result is the exact distribution. Monte Carlo and partial branching are also available for models whose trees are too large.

What changes in your model. One thing: every random value is obtained with

```
double t = Para.choose("service", Law.of(Arrays.asList(0.5, 1.5), new double[]{0.6,
    0.4}));
```

instead of `Math.random()` or `new Random()`. The model is otherwise an ordinary MS4 model. When no engine is running, as in MS4 Me or a plain `SimulationImpl` run, `Para.choose` simply samples, so the same model still works as a normal stochastic simulation.

How the engine runs it. Figure 1 shows the cycle. It follows the algorithm in `main_paratemporal.tex` (MAIN, Γ_{PT}, ADVANCE, DOEXPANSION).

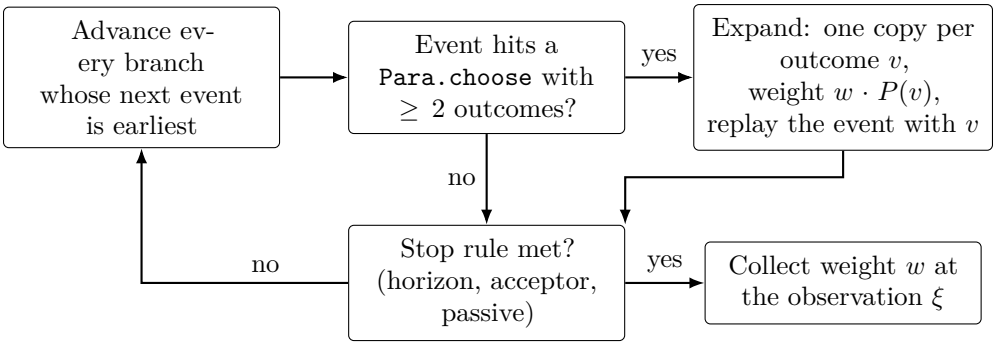


Figure 1: The engine cycle. All branches share one simulation clock. Branching takes no simulated time.

What you get back. A `ParaResult`: the weighted distribution of terminal observations (`distribution()`), expectations of any function of the observation (`expectation(f)`), distributions of a summary (`marginal(g)`), the omitted probability when you prune (`residual()`), and run statistics (`stats()`).

2 Setup

2.1 Requirements

- JDK 17 or newer (`java` and `javac` on the `PATH`); Git Bash for the scripts.

- The MS4 DEVS core. Two sources work:
  - the core *sources* bundled in `Binghamton/Demo Deliverable/ParaDEVS/src/com`, which need no MS4 Me installation and are used by default, or
  - the MS4 Me 1.5 jars in `C:\Program Files\MS4 Systems\MS4 Me\plugins`, which are needed for code generated by MS4 Me (Section 5).

2.2 Folder layout

Path (in `ParaDEVS_Library/`)	Contents
`src/main/java/paradevs/`	Model API: `Para.choose`, `Law`
`src/main/java/paradevs/engine/`	Engine, `ParaSpec`, `ParaResult`, selection rules, input trace, transducer
`src/main/java/paradevs/ms4/`	MS4 integration: coordinator, snapshots, merge keys
`src/examples/java/examples/`	`tutorial/`, `reward/`, `finance/`, `baseball/` with their tests
`src/sandbox/java/sandbox/`	Staging area for your experiments (template: `RandomWalk`, `RunSandbox`)
`lib/ms4-core.jar`	MS4 core packaged from Demo Deliverable, used by VS Code
`.vscode/`	VS Code settings, run configurations and tasks
`src/ms4me_import_demo/java/`	JobProcessorExample imported from MS4 Me (Section 5)
`scripts/`	`build_library.sh`, `run_my_model.sh`, `run_ms4me_import_demo.sh`
`notes/`	Design notes; `05_library_v0.md` describes the implementation
`docs/howto/`	This guide

2.3 Build and run the tests

From the `ParaDEVS_Library` folder:

```
bash scripts/build_library.sh             # build, then run every test
bash scripts/build_library.sh tutorial    # one test: sandbox | tutorial | reward |
    finance | baseball | plain
```

Each test prints `PASS`/`FAIL` lines and a tally. The full suite currently passes 77 checks: the tutorial against a hand answer, the paper's timed reward example against exact rational results, finance against the Demo Deliverable baseline, and baseball against the original BaseballDEVS_v7 engine.

2.4 Run your own model

Put your `.java` files in any folder (any package layout) and give the class that has `main`:

```
bash scripts/run_my_model.sh path/to/my/src  mymodels.RunMyModel           # bundled
    core
bash scripts/run_my_model.sh path/to/my/src  mymodels.RunMyModel --ms4me   # MS4 Me
    jars
```

The script compiles the MS4 core, the library and your folder together into `build/user`, then runs your class.

2.5 Working in VS Code

VS Code is the current environment for testing. Open the `ParaDEVS_Library` folder itself (File → Open Folder) with the Java Extension Pack installed. The folder is preconfigured:

- **Sources and classpath** (`.vscode/settings.json`): `src/main/java`, `src/examples/java` and `src/sandbox/java`, plus `lib/ms4-core.jar`. Exactly this configuration compiles cleanly from the command line. If the editor shows errors on first open, run "Java: Clean Java Language Server Workspace".
- **Run and Debug** (`.vscode/launch.json`): *Sandbox: RunSandbox*, *Run current file*, the tutorial, each `Verify*` test, and a plain MS4 run. All run with the library folder as the working directory, so relative data paths work exactly as in the scripts. Each target was run with this classpath and working directory, and all pass. Use the Debug variant to set breakpoints in model code or in the engine.
- **Tasks** (Terminal → Run Task, run in Git Bash): build and run all tests, run one test, package the MS4 core jar, the MS4 Me import demo, and compile this guide.

Where to put new work: `src/sandbox/java/sandbox/`. The template there is a random walk whose next step is drawn on state entry. `RunSandbox` checks it against the binomial law and shows safe merging shrinking 64 leaves to 12. Copy the two files, rename them, and start from there. When a model is worth keeping, move it under `src/examples` with a test.

Note. The MS4 Me import demo is not on the VS Code source path, because it needs the MS4 Me jars, which would clash with `lib/ms4-core.jar`. Run it with its task. If the Demo Deliverable core sources change, rerun the *package MS4 core jar* task.

3 Key ideas in one page

Term	Meaning
Draw (choice point)	A call to `Para.choose(label, law)` with at least two possible outcomes. Laws with one outcome never branch.
`Law`	A finite list of (value, probability) pairs. It must sum to 1 (within 10^{-6}). Zero-probability values are dropped.
Departure	One DEVS event of the top model: internal, external (input), or confluent (both). Draws happen inside departures, including in `initialize()`.
Branch	One copy of the run: model state, clock, position in the input trace, entry count, observation ξ, weight w.
Persistent entry	A completed state after a departure. The initial state is entry 1.
Input trace	A fixed list of timed messages sent to the top model's input ports.
Transducer	Folds the top model's *outputs* into the observation ξ (e.g. "number of jobs done"). ξ must be immutable with `equals`/`hashCode`.
Stop rules	A branch ends at the *horizon* (entry count m), when an *acceptor* returns true, or when it is *passive* (no event scheduled and no inputs left).
Selection rule	What to do at a draw: *exhaustive* (all outcomes), *Monte Carlo* (one sampled outcome), *partial* (outcomes above a threshold), or *hybrid* (a rule per label).
Safe merge	Two new branches with identical complete state and identical ξ are combined, and their weights added. This never changes the result.
Residual	Probability left unexplored by partial selection. True probabilities lie between the result and result + residual.

There is no "step". The old tree engines worked in steps: one tree level, one outcome forced in from outside, one simulation run, one state save, all at once. The library separates these. Time advances one DEVS event (departure) at a time. Branching happens only where the model calls `Para.choose`. An event may make no draw (no branching), one draw, or several, and a later draw may depend on an earlier one in the same event. There is no `depth`: a run ends by its stop rules. See the **Architecture Guide**, Section "Step versus draw".

Note. The engine observes only what leaves the top model through its output ports. If you care about internal state, such as cash or a queue length, have a component report it on an output port and couple that port out (both tutorials do this).

The one rule for models. Given the state, the event, and the values returned by `Para.choose`, a model's transition must be *deterministic*. The engine replays an event with the chosen values to continue a branch, so the model must ask for the same draws in the same order. Do not use `Math.random`, `Random`, wall-clock time, or the iteration order of `HashSet`/`HashMap` over objects without `equals`.

4 Tutorial 1: a DEVS model built purely in Java

We build a single server with no waiting room. Customers arrive at times 1, 2 and 3. The server takes 0.5 time units with probability 0.6 or 1.5 with probability 0.4. A customer who arrives while the server is busy is lost. Question: *how many customers are served?* The whole model is in `src/examples/java/examples/tutorial/`.

4.1 The answer by hand

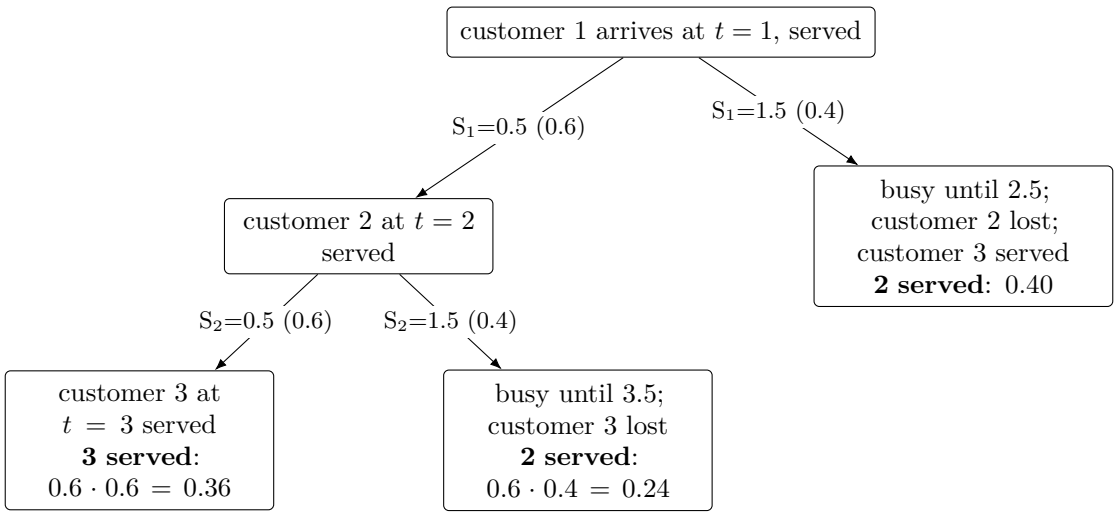


Figure 2: The tree the engine will explore. Customer 3's own service time also branches, but does not change the count.

So $P(3 \text{ served}) = 0.6 \times 0.6 = 0.36$, $P(2 \text{ served}) = 0.64$, and $E[\text{served}] = 2.36$. The tree has 5 leaves: customer 3's service time adds a split in two places.

4.2 Step 1: write the atomic model

An MS4 atomic model extends `AtomicModelImpl` and overrides the four DEVS functions: `getTimeAdvance` (ta), `getOutput` (λ), `internalTransition` (δ_{int}) and `externalTransition` (δ_{ext}). `confluentTransition` defaults to internal, then external. Ports are created with `addInputPort`/`addOutputPort`. The only paratemporal-specific code is the `SERVICE` law and the `Para.choose` call in `externalTransition`.

Listing 1: `Server.java`: the complete atomic model

```
package examples.tutorial;

import java.util.Arrays;

import com.ms4systems.devs.core.message.MessageBag;
import com.ms4systems.devs.core.message.Port;
import com.ms4systems.devs.core.message.impl.MessageBagImpl;
import com.ms4systems.devs.core.model.impl.AtomicModelImpl;
```

```java
import paradevs.Law;
import paradevs.Para;

/**
 * Tutorial atomic model: a single server with no queue.
 * idle --customer--> busy for a random service time (0.5 w.p. 0.6, 1.5 w.p. 0.4) -->
    emits done, idle.
 * A customer arriving while busy is lost; the remaining service time is kept.
 */
public class Server extends AtomicModelImpl {
    private static final long serialVersionUID = 1L;

    /** The only paratemporal-specific line: the service-time law. */
    static final Law<Double> SERVICE = Law.of(Arrays.asList(0.5, 1.5), new
    double[]{0.6, 0.4});

    public final Port<Integer> inCustomer = addInputPort("customer", Integer.class);
    public final Port<Integer> outDone = addOutputPort("done", Integer.class);

    protected String phase;
    protected int current;
    protected double sigma;

    public Server() { super("Server"); }

    @Override public void initialize() {
        super.initialize();
        phase = "idle"; current = -1; sigma = Double.POSITIVE_INFINITY;
    }

    @Override public Double getTimeAdvance() { return sigma; }

    @Override public MessageBag getOutput() {
        MessageBag out = new MessageBagImpl();
        if (phase.equals("busy")) out.add(outDone, current);
        return out;
    }

    @Override public void internalTransition() {
        phase = "idle"; current = -1; sigma = Double.POSITIVE_INFINITY;
    }

    @Override public void externalTransition(double e, MessageBag input) {
        int id = inCustomer.getData(input).get(0);
        if (phase.equals("idle")) {
            current = id;
            sigma = Para.choose("service", SERVICE);   // the engine branches here
            phase = "busy";
        } else {
            sigma -= e;                                 // lost customer; keep serving
        }
    }
}
```

Points to notice:

- State lives in fields (`phase`, `current`, `sigma`) and is set in `initialize()`, which the engine calls at time 0. Do not rely on field initializers for state that `initialize()` should reset.
- `sigma -= e` keeps the remaining service time when a customer is lost. That is standard MS4 style.
- Fields must be `Serializable`: the engine copies branches by serialization. Primitives, strings, arrays and your own `Serializable` classes are fine.

4.3 Step 2: run it with an input trace

To run the server alone, customers come from an input trace. The port name in the trace is the name given to `addInputPort` (here `"customer"`). The transducer counts `done` outputs; that count is the observation.

Listing 2: Transducer and the first engine run (from `RunServiceLine.java`)

```
    /** xi = number of customers served so far (a sufficient summary, so branches can
    merge). */
    static final Transducer<Integer> SERVED = new Transducer<Integer>() {
        @Override public Integer initial() { return 0; }
        @Override public Integer record(Integer served, double t, List<OutputEvent>
    outputs) {
            for (OutputEvent e : outputs) if (e.port.equals("done")) served++;
            return served;
        }
    };
        InputTrace arrivals = InputTrace.builder()
            .at(1.0, "customer", 1).at(2.0, "customer", 2).at(3.0, "customer",
    3).build();
        ParaResult<Integer> atomic = ParaEngine.run(ParaSpec.of(Server::new,
    SERVED).trace(arrivals));
        print("Step 1  Server + input trace", atomic);
```

Output:

```
Step 1  Server + input trace
    served=2  status=PASSIVE  weight=0.6400
    served=3  status=PASSIVE  weight=0.3600
    E[served]=2.3600  mass=1.0000  leaves=5 choiceNodes=4 departures=23 merged=0
    maxLive=5 ms=68
```

That is the hand answer, with 5 leaves and 4 draws expanded. `PASSIVE` means each branch ended because the server was idle and no inputs were left, so no horizon was needed.

4.4 Step 3: make it a coupled model

Now generate the customers inside the model. `Arrivals` is a deterministic atomic model, and `ServiceLine` couples the two. This is exactly what an SES would declare.

Listing 3: `Arrivals.java` (class body)

```java
/** Tutorial atomic model: emits customers 1..n at times 1, 2, ..., n, then stops.
    Deterministic. */
public class Arrivals extends AtomicModelImpl {
    private static final long serialVersionUID = 1L;

    public final Port<Integer> outCustomer = addOutputPort("customer", Integer.class);

    protected final int n;
    protected int next;
    protected double sigma;

    public Arrivals(int n) { super("Arrivals"); this.n = n; }

    @Override public void initialize() { super.initialize(); next = 1; sigma = 1.0; }

    @Override public Double getTimeAdvance() { return sigma; }

    @Override public MessageBag getOutput() {
        MessageBag out = new MessageBagImpl();
        out.add(outCustomer, next);
        return out;
    }

    @Override public void internalTransition() {
        next++;
        sigma = next > n ? Double.POSITIVE_INFINITY : 1.0;
    }
}
```

Listing 4: `ServiceLine.java`: the coupled model

```java
/**
 * Tutorial coupled model, equivalent to the SES
 *   ServiceLine is made of Arrivals and Server!
 *   Arrivals sends customer to Server!
 *   Server sends done to ServiceLine!
 */
public class ServiceLine extends CoupledModelImpl {
    private static final long serialVersionUID = 1L;
    public final Port<Integer> outDone;

    public ServiceLine(int customers) {
        super("ServiceLine");
        Arrivals arrivals = new Arrivals(customers);
        Server server = new Server();
        addChildModel(arrivals);
        addChildModel(server);
        outDone = addOutputPort("done", Integer.class);
        addCoupling(arrivals.outCustomer, server.inCustomer);
        addCoupling(server.outDone, outDone);
    }
}
```

The engine runs coupled models with the same call; no trace is needed now:

```
ParaResult<Integer> coupled = ParaEngine.run(ParaSpec.of(() -> new ServiceLine(3),
    SERVED));
```

```
Step 2  ServiceLine (coupled)
    served=2  status=PASSIVE  weight=0.6400
    served=3  status=PASSIVE  weight=0.3600
    E[served]=2.3600  mass=1.0000  leaves=5 choiceNodes=4 departures=23 merged=0
    maxLive=5 ms=33
```

Note. Pass a *factory* (`() -> new ServiceLine(3)` or `Server::new`), not a model instance. The engine builds the model once, then copies it.

4.5 Step 4: options

```
ParaSpec.of(() -> new ServiceLine(3), SERVED).merge(ParaSpec.Merge.SAFE);         //
    safe merging
ParaSpec.of(() -> new ServiceLine(3), SERVED)
        .selection(SelectionRule.monteCarlo()).roots(2000).seed(11);               //
    Monte Carlo
ParaSpec.of(() -> new ServiceLine(3), SERVED)
        .selection(SelectionRule.partialExhaustive(0.5));                          //
    prune outcomes < 0.5
```

```
Step 4  ServiceLine, Monte Carlo N=2000
    served=2  status=PASSIVE  weight=0.6465
    served=3  status=PASSIVE  weight=0.3535
    E[served]=2.3535  mass=1.0000  leaves=2000 choiceNodes=4707 departures=17414 ...
```

Monte Carlo gives an unbiased estimate, here within one standard error of 0.36. Safe merging finds nothing to merge in this model, because no two branches have the same state at the same time. In the baseball example it cuts 24,450 leaves to 195.

4.6 Step 5: it is still an ordinary MS4 model

```
Para.setDefaultRandom(new Random(3));      // optional: reproducible sampling
Simulation sim = new SimulationImpl("plain run", new ServiceLine(3));
sim.startSimulation(0);
sim.simulateIterations(Long.MAX_VALUE);
```

With no engine active, `Para.choose` samples, and MS4 runs one random path.

4.7 Step 6: test it

A model is ready when these hold. `RunServiceLine` checks all of them:

1. Exhaustive `totalMass()` is 1 (up to rounding in your data).
2. The result matches a case you can work out by hand (as in Figure 2).
3. Safe merging gives the same distribution as no merging.

4. Monte Carlo agrees with exhaustive within about 3–4 standard errors, $\sqrt{p(1-p)/N}$.
5. The plain MS4 run completes.

Run it with `bash scripts/build_library.sh tutorial`; it ends with `10 passed, 0 failed`.

Try next. Change `ServiceLine(3)` to `ServiceLine(8)` and watch the leaf count grow. Add a second outcome law in `Arrivals` (random inter-arrival times). Or add an output port reporting lost customers and count both in the transducer.

5 Tutorial 2: bringing a model over from MS4 Me

MS4 Me turns DNL into Java atomic models and turns SES/PES into a Java coupled model. The engine runs that Java directly. This section uses the `JobProcessorExample` project from the MS4 Me workspace; the result is in `src/ms4me_import_demo/java`.

5.1 What to bring over

From the MS4 Me project	What it is	Needed?
`src/Models/java/<Atomic>.java`	Java generated from each `.dnl`	**yes**
`src/Models/java/<Coupled>.java`	Coupled model written by the PES (`write class files`); its `make()` holds `addChildModel`/`addCoupling`	**yes** (or write the couplings yourself)
`src/Models/java/<Data>.java`	Message and data classes (e.g. `WorkToDo`, `Players`)	**yes**
`src/Models/txt/*.csv`	Input data the models read	if used
`src/Models/dnl, ses, pes`	Specifications	no (keep them for regeneration)
`Run*, GeneralTree, GeneralNode, SimulateModel, SimResult`	The old hand-built tree engine	**no**: the library replaces it

Keep the package name (`Models.java`) as generated, so you don't have to edit every file.

5.2 Pick a build route

Generated MS4 Me files import `SimViewer` and `SimulationOptionsImpl` for their `main()` methods. Those two classes are in the MS4 Me jars but not in the bundled core.

Route	Use when	Command
A. MS4 Me jars	MS4 Me is installed; files stay as generated	`run_my_model.sh <dir> <Main> --ms4me`
B. Bundled core	No MS4 Me installation; you can edit the files	delete each generated `main()` and its `SimViewer`/`helpers` imports, then `run_my_model.sh <dir> <Main>`

Route B is what the Demo Deliverable did: its generated `main()` methods are commented out.

5.3 The edits

E1. Replace every random draw by `Para.choose`. Search for `Math.random`, `new Random`, `nextDouble`, `nextInt`, and any sampling helper. Inside DNL code blocks, use fully qualified names so no import is needed:

```
paradevs.Para.choose("processingTime",
    paradevs.Law.of(java.util.Arrays.asList(5.0, 15.0), new double[] {0.7, 0.3}))
```

E2. Expose what you want to measure. The engine sees only the top model's outputs. Add an output port to the coupled model and couple the interesting component port to it (`addCoupling(child.outX, addOutputPort("x", T.class))`).

E3. Move the starting state into `initialize()`. The old engines injected state by reflection (`setValue(cm, "Portfolio", "Shares", ...)`, `Portfolio.phase = cState`). Under the library, the model starts from `initialize()` and evolves itself. Constructor parameters are fine for fixed data such as tables or prices.

E4. Move forced outcomes inside the model. The old pattern set an outcome from outside (`setValue(cm,"HitMaker","HitType",outcome,...)`) and sent its probability through the message. Instead, the component that knows the situation draws the outcome itself:

```
// before (HitMaker): outcome injected, probability sent along with it
//   if (HitType == "Single") { Prob = batter.getBattingProb()[1];
    output.add(outSingle, ...); }
// after (Batter): the batter draws the outcome
hit = Para.choose("hit", Law.of(HITS, scoringPosition ? scoring[index] :
    normal[index]));
```

If the law depends on another component's state (bases occupied, shares held), have that component report it on a coupling, as the finance and baseball examples do.

E5. Data classes: make them `Serializable`. Give message and observation classes `equals`/`hashCode` if you want safe merging to find matches.

5.4 Worked example: JobProcessorExample

The generator gives each job a processing time of `20 + count`. We make it 5 or 15 at random and observe solved jobs. These are the only two edits in the generated files:

```
// GeneratorOfJobs.java (in initialize() and in the internal transition)
```

```
// ParaDEVS: random processing time (was: wtd.setProcessingTime(20 + count);)
wtd.setProcessingTime(paradevs.Para.choose("processingTime",
    paradevs.Law.of(java.util.Arrays.asList(5.0, 15.0), new double[] {0.7, 0.3})));

// SimpleWorkFlow.java, end of make()
// ParaDEVS: expose solved jobs as a coupled-model output so the experiment can
    observe them
addCoupling(ProcessorOfJobs.outJob, addOutputPort("solved", WorkToDo.class));
```

The driver counts `solved` outputs and stops each branch at its first event at or after $t = 45$ (an acceptor, since the generator would otherwise run for 50 jobs):

Listing 5: Driver (from `demo/RunJobShop.java`)

```
    /** xi = number of jobs solved so far. */
    static final Transducer<Integer> SOLVED = new Transducer<Integer>() {
        @Override public Integer initial() { return 0; }
        @Override public Integer record(Integer n, double t, List<OutputEvent> outs) {
            for (OutputEvent e : outs) if (e.port.equals("solved")) n++;
            return n;
        }
    };

    public static void main(String[] args) {
        // Stop each branch at its first event at or after t = 45.
        ParaSpec.Acceptor<Integer> byTime45 = (xi, t, n) -> t >= 45;

        ParaResult<Integer> exact = ParaEngine.run(ParaSpec.of(SimpleWorkFlow::new,
    SOLVED).acceptor(byTime45));
        print("Exhaustive", exact);
```

Run it with `bash scripts/run_ms4me_import_demo.sh`:

```
Exhaustive: leaves=42 choiceNodes=41 departures=163 merged=0 maxLive=32 ms=651
    solved by t=45: 2   P=0.2160
    solved by t=45: 3   P=0.5439
    solved by t=45: 4   P=0.2401
    E[solved]=3.0241  mass=1.0000
Monte Carlo N=2000: leaves=2000 ...
    solved by t=45: 2   P=0.2160
    solved by t=45: 3   P=0.5465
    solved by t=45: 4   P=0.2375
Plain MS4 run completed 200 iterations.
```

Check: four jobs are solved by $t = 45$ only if all four processing times are short, $0.7^4 = 0.2401$. The first job's time is drawn inside `initialize()`, and the engine branches there too.

5.5 Port names in generated code

DNL `accepts input on Job` creates a port named `inJob`, and `generates output on Job` creates `outJob`. Use those names in input traces and transducers (`InputTrace.builder().at(5.0, "inJob", job)`).

5.6 Keeping MS4 Me in sync

If you regenerate Java from DNL, edits made only in the Java are lost. Put the `Para.choose` call in the DNL code block (fully qualified, as in E1), and copy `src/main/java/paradevs` into the MS4 Me project's `src` folder so the generated code compiles there.

Note. The two steps in this subsection have not yet been tried inside the MS4 Me IDE. Everything else in this section was run as shown.

5.7 Testing an imported model

Use the checklist of Tutorial 1, Step 6, plus one more test when an old result exists. The finance and baseball tests are complete templates (`src/examples/java/examples/finance/VerifyFinance.java`, `.../baseball/VerifyBaseball.java`). The approach:

- Run the old project once and save its output (a CSV of probabilities or expected values).
- Run the library exhaustively and map each terminal observation to the old summary with `marginal(g)`, e.g. shares held, or (bases, outs, runs).
- Compare probabilities to about 10^{-12} and money or runs to about 10^{-6}.

The old IH engines merged branches by a summary and added their weights. Because that merge is linear, the library's exact distribution reproduces those numbers through `marginal`, without any IH merging.

6 API reference

6.1 Model side

Call	Use
`Para.choose(String label, Law<T> law)`	Draw a value. The label names the primitive (used by hybrid selection and in error messages).
`Law.of(v1, p1, v2, p2, ...)`	Law from value/probability pairs. Avoid it when values are numbers too; use the list form.
`Law.of(List<T> values, double[] probs)`	Law from parallel arrays.
`Law.of(Map<T, Number> probs)`	Law from an ordered map.
`Law.dirac(v)`	A single certain value (never branches).
`Law.asGiven(values, probs, tol)`	Accept a total mass within `tol` of 1, to reproduce slightly improper source data.
`Para.setDefaultRandom(Random r)`	Seed the sampler used outside the engine.

6.2 Experiment side: `ParaSpec`

Builder call	Meaning (default)
`ParaSpec.of(factory, transducer)`	Model factory and observation (required).
`.trace(InputTrace)`	Fixed timed inputs (none).
`.horizon(m)`	Stop at the m-th persistent entry; the initial state is entry 1 (no limit).
`.acceptor((xi, t, n) -> boolean)`	Extra stopping rule on observation, time, entry count (none).
`.selection(SelectionRule)`	`exhaustive()` (default), `monteCarlo()`, `partialExhaustive(p)`, `hybrid(map, other)`.
`.roots(N)`	N starting copies of weight $1/N$; use with Monte Carlo (1). Not combinable with safe merging (refused).
`.merge(ParaSpec.Merge.SAFE)`	Merge identical new branches (`NONE`).
`.seed(s)`	Seed for sampled selection (1).
`.maxLive(n)`	Abort if more than n branches are live (5,000,000).

`InputTrace.builder().at(time, port, value)...build()`: messages at the same time are delivered together. `Transducer.history()` records every output with its time; write your own (Tutorial 1) to keep a smaller summary, which also merges better.

6.3 Results: `ParaResult`

Method	Returns
`distribution()`	Map from (observation ξ, stop status) to total weight.
`expectation(f)`	$\sum w\, f(\xi)$: expected value of a function of the observation.
`marginal(g)`	Distribution of $g(\xi)$, with weights of equal images summed.
`totalMass()`	Sum of weights; 1 for exhaustive runs with proper laws.
`residual()`	Omitted probability from partial selection.
`stats()`	`leaves` (finished branches), `choiceNodes` (draws expanded), `departures` (event runs incl. replays), `merged`, `maxLive`, `ms`.

7 Troubleshooting

Message or symptom	Cause and fix
`Law mass must be 1 (got ...)`	Probabilities do not sum to 1. Fix the data, or use `Law.asGiven(..., tol)` on purpose.
`Replay mismatch at draw '...'` or `Replay used k of m supplied values`	The event is not deterministic given its state: other randomness, time-of-day, or hash-order iteration. Make all randomness go through `Para.choose`.
`choose() called after a suspension`	Model code catches `Throwable`/`Error` around a draw. Let `Para.Suspension` pass through.
`Model state is not serializable`	A field's class is not `Serializable`. Make it so. Use `transient` only for caches that can be rebuilt.
`Model ... has no input port ...`	Trace port name differs from the model's port. Generated ports are named `inX`/`outX`.
`Live branches exceed maxLive`	The tree is too large. Use a horizon or acceptor, safe merging, Monte Carlo, or partial selection.
Result empty or every observation `null`	Nothing reaches the top model's outputs. Couple the port out (E2) and check the transducer's port name.
Merging never merges	ξ lacks `equals`/`hashCode`, ξ is a full history, or the state really differs (e.g. different times).
`Model failed during INIT ...`	An exception in model code. Note that `initialize()` runs at time 0.
Very slow run	Count `departures`: each draw replays its event. Merge, shorten the horizon, or sample.

8 Scope of v0 and further reading

- Supported: atomic and coupled MS4 models, fixed input traces, horizon/acceptor/passive stopping, exhaustive, Monte Carlo, partial and hybrid selection, safe merging.
- Not yet: deferred output draws (the paper's *ps* representation), inputs that react to outputs (e.g. an RL agent), parallel execution, exhaustive treatment of continuous laws (sample them instead).
- The engine follows `Do s/documents/paratemporal/main_paratemporal.tex`; the replay mechanism is the "derandomized departure" of `Paratemporal_Adaptation.tex`. Implementation notes and the verification record are in `notes/05_library_v0.md`.